

 FUNCTIONAL TEST SUITE SPECIFICATION

# TicketWave — Functional Test Cases

A detailed breakdown of automated end-to-end booking & payment test scenarios, idempotency guarantees, race condition guards, and coverage gates.



SECTION 1

# Test Coverage & Execution Summary

---

670 passing tests with 0 failures, backed by real PostgreSQL Testcontainers integration testing.



# Functional Suite Coverage Summary

| Flow Domain                  | Unit Test Class (Cases)                     | Integration Test Class (PostgreSQL)    | Controller MockMvc Class                |
|------------------------------|---------------------------------------------|----------------------------------------|-----------------------------------------|
| Booking Creation & Lifecycle | <code>BookingServiceImplTest</code> (44)    | <code>BookingFlowIT</code> (4)         | <code>BookingControllerTest</code> (19) |
| PNR Generation               | <code>PnrGeneratorImplTest</code> (3)       | —                                      | —                                       |
| Seat Hold & Expiration       | <code>SeatHoldServiceImplTest</code> (21)   | <code>SeatHoldConcurrencyIT</code> (1) | —                                       |
| Payment & Idempotency        | <code>PaymentServiceImplTest</code> (21)    | <code>PaymentFlowIT</code> (5)         | via <code>BookingControllerTest</code>  |
| Reschedule & Settlement      | <code>RescheduleServiceImplTest</code> (17) | —                                      | via <code>BookingControllerTest</code>  |
| Refunds & Policy             | <code>RefundServiceImplTest</code> (26)     | <code>RefundFlowIT</code> (8)          | <code>RefundControllerTest</code> (4)   |



# | Booking Creation & Idempotency Rules



## Idempotency Key Deduplication

**Replayed Key:** Submitting an identical `idempotencyKey` returns the original booking without re-holding seats ( `BK-02` ).

**DB Race Resolution:** Concurrent inserts with the same key throw `DuplicateBookingRequestException` ( `BK-03` ).



## Promos & Validation

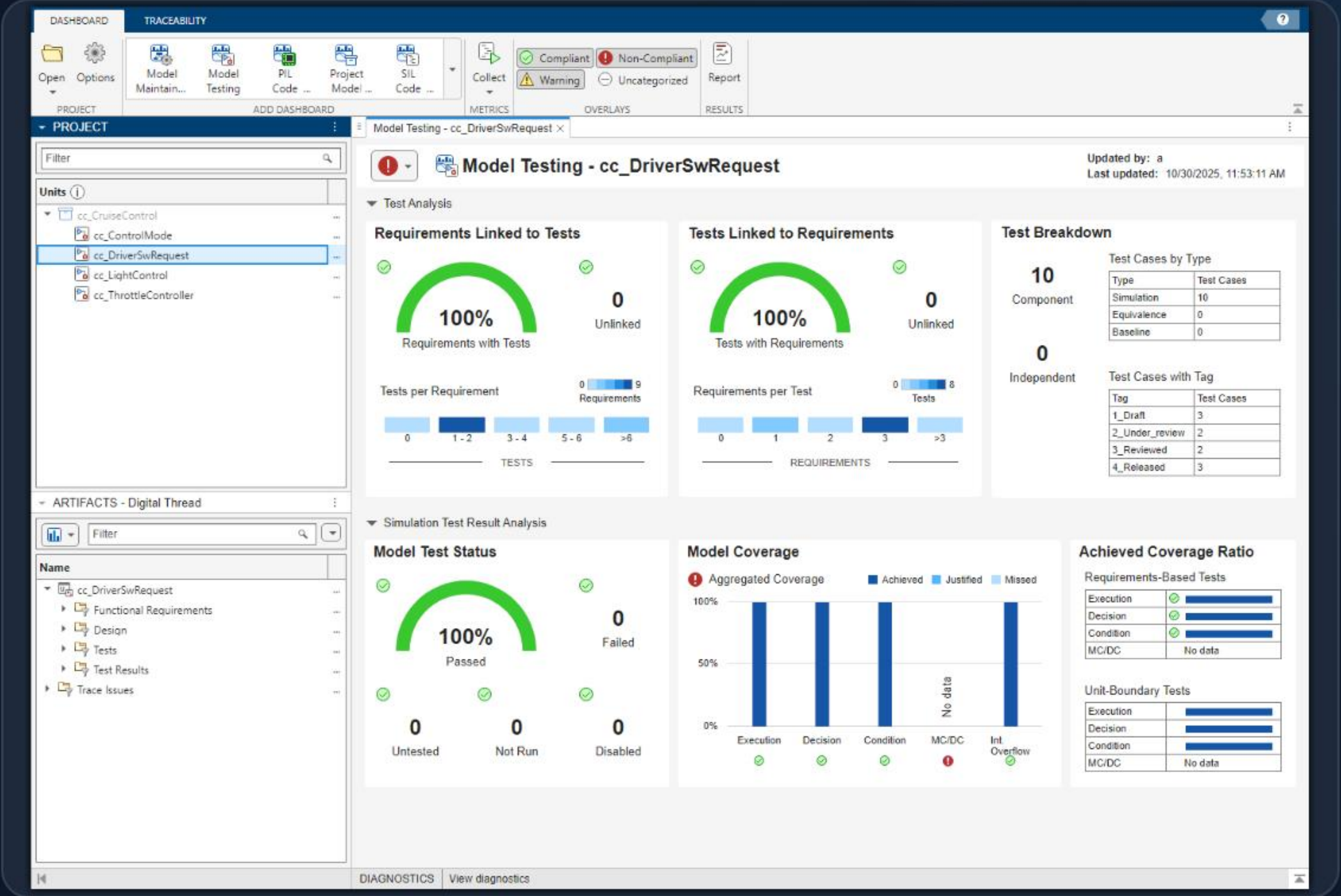
**Promo Code Application:** Valid codes apply discounts and record on booking; blank strings are treated as null without errors ( `BK-04/05` ).

**Defensive Isolation:** Referencing passengers owned by another account throws `PassengerNotFoundException` ( `BK-07` ).



# Concurrency & Deadlock Prevention

-  **Ascending ID Lock Ordering:** Seats are locked strictly in ascending ID order, preventing lock cycles across racing checkouts ( [BK-01](#) ).
-  **Opposite Order Concurrency Test:** Tested in [BookingFlowIT.java:173](#) — simultaneous requests over overlapping seats neither deadlock nor double-book ( [BK-09](#) ).
-  **Database Transaction Truth:** Real PostgreSQL Testcontainers verify pessimistic row locks under multi-threaded contention ( [BK-10](#) ).





# | Seat Hold & Expiration Mechanics



## 1. TTL Hold & Re-affirmation

Holds seat with expiration. Submitting a new hold by the same owner extends the TTL rather than throwing an error ( [SH-03](#) ).



## 2. Privacy-Safe Release

Releasing a hold owned by someone else is an idempotent **silent no-op**, preventing state discovery ( [SH-06](#) ).



## 3. Bulk Sweeper & Expired Reclaim

Expired holds are reclaimed on access or swept in bulk by `releaseExpiredHolds` without locking the table ( [SH-09](#) ).



# | Payment Idempotency & 3-D Secure



## Payment Idempotency & Retries

**Reference Replay:** Replayed payment references return the original payment result without re-charging ( `PAY-01/07` ).

**Decline & Retry:** A declined card leaves the seat hold intact, allowing the customer to retry with a valid card ( `PAY-06` ).



## 3-D Secure Challenge Workflow

**Pending Challenge:** Cards requiring 3DS save payment as `PENDING_3DS` without settling the booking ( `PAY-12` ).

**Challenge Settlement:** Valid challenge code confirms payment & booking; invalid code declines payment ( `PAY-13` ).



# Reschedule & Fare Settlement

## Side-Effect-Free Quotes

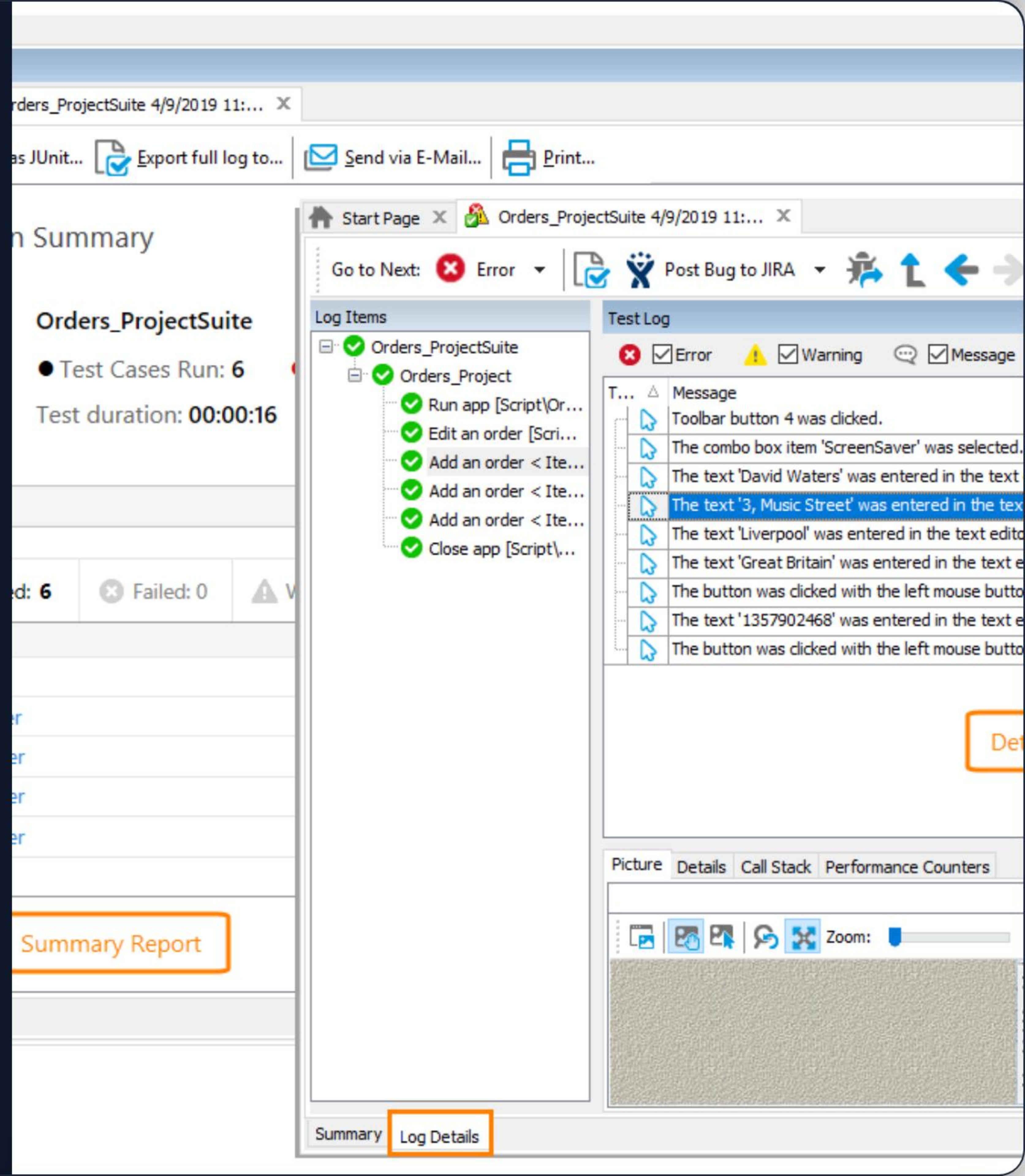
`previewReschedule` calculates fare differences and proximity rules without mutating booking state ( `RS-01/02` ).

## Fare Top-ups & Credit Refunds

Upgrades collect top-up payments; downgrades reduce payment total and issue a `RESCHEDULE_CREDIT` refund ( `RS-07/09` ).

## Proximity Guarding

Reschedule requests submitted too close to departure throw `CancellationNotAllowedException` ( `RS-10` ).





# | Refund Lifecycle & Agent Overrides



## 1. Customer Initiation

Creates a **PENDING** refund. Booking stays **CONFIRMED** and seats remain booked during support review ( **RF-01** ).



## 2. Support Approval

Marks refund **PROCESSED**, refunds payment, cancels booking, and releases seats for resale ( **RF-07** ).



## 3. Agent Fee Waiver

Support agents can apply fee waivers with a mandatory reason, persisting the waived delta ( **RF-09** ).



✔ TICKETWAVE FUNCTIONAL TESTS VERIFIED

# Questions & Discussion

670 Tests Passing | 100% Financial Module Coverage Gate Enforced

```
cd backend && mvn clean verify
```



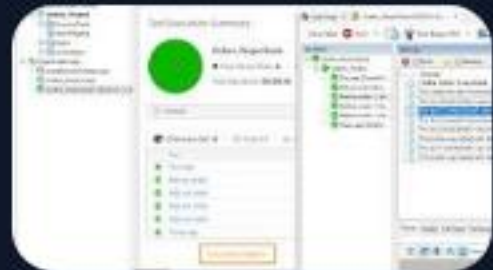
# | Image Sources



<https://au.mathworks.com/help/slcheck/ref/model-testing-dashboard-complete.png>

Source: [www.mathworks.com](https://www.mathworks.com)

---



[https://static1.smartbear.co/smartbearbrand/media/images/product/testcomplete/testreportinganalysis\\_mainbanner.png](https://static1.smartbear.co/smartbearbrand/media/images/product/testcomplete/testreportinganalysis_mainbanner.png)

Source: [smartbear.com](https://smartbear.com)